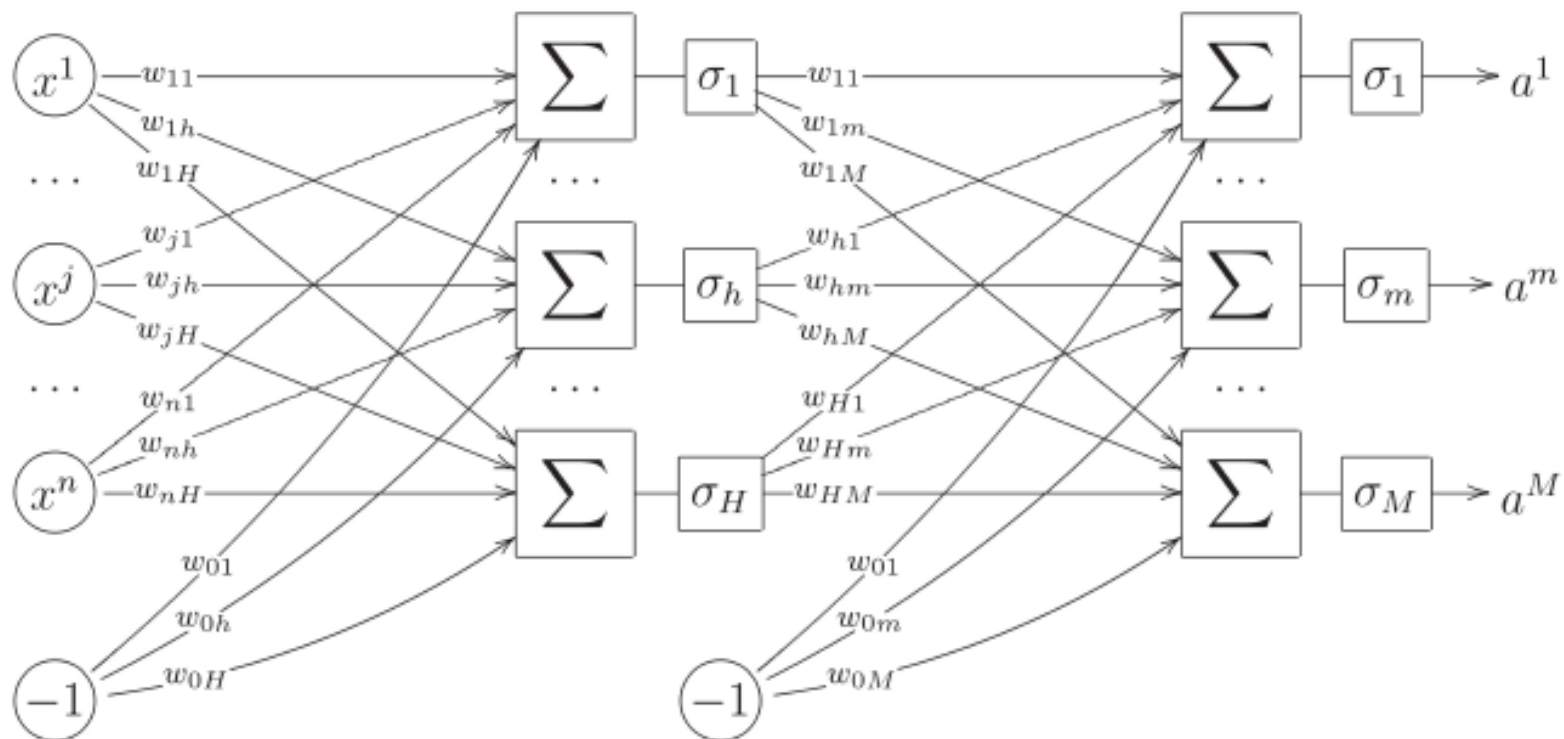


Метод обратного распространения ошибки. Тонкости обучения и его улучшения

входной слой,
 n признаков

скрытый слой,
 H нейронов

выходной слой,
 M нейронов



Многослойная нейронная сеть

Среди многослойных нейронных сетей можно выделить четыре наиболее значимых и важных класса нейронных сетей:

- ▶ **сети прямого распространения** — все связи направлены строго от входных нейронов к выходным (многослойный персептрон), только один скрытый слой;
- ▶ **рекуррентные нейронные сети или сети обратного распространения** — сигнал в таких сетях с выходных нейронов или нейронов скрытого слоя частично передается обратно на входы нейронов входного слоя;
- ▶ **радиально базисные функции** — вид многослойной нейронной сети, имеющий скрытый слой из радиальных элементов и выходной слой из линейных элементов. Сети этого типа довольно компактны и быстро обучаются.
- ▶ **самоорганизующиеся карты или сеть Кохонена**. Такой класс успешно применяется в задачах распознавания. Сети такого класса способны выявлять новизну во входных. Сеть Кохонена имеет всего два слоя: входной и выходной, составленный из радиальных элементов.

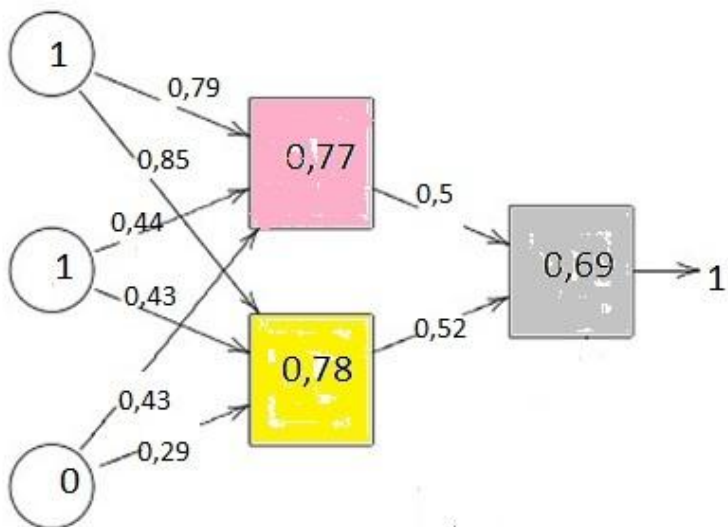
Задача обучения нейронной сети

- ❑ **Цель обучения** – подобрать веса нейронной сети наилучшим образом
- ❑ **Задача обучения** – задача минимизации **функции ошибки** или **функция стоимости** (cost function), отражающей разницу ожидаемого сигнала, полученного на выходе сети и фактического сигнала, соответствующего текущему входу, по всей обучающей выборке

Backpropagation. Нюансы обучения

Пример: Рассмотрим сеть которая должна выдавать ответ: «идти на вечеринку или нет» (нужный ответ «нет», т.е. 0). Сигналы, поступающие на входной слой нейронов следующие:

1. Имеется ли подходящий костюм (1)
2. Хорошая ли погода (1)
3. Будет ли друг (0)



Вектор весов задаем случайным образом.

Подсчитаем значения в скрытых нейронах:

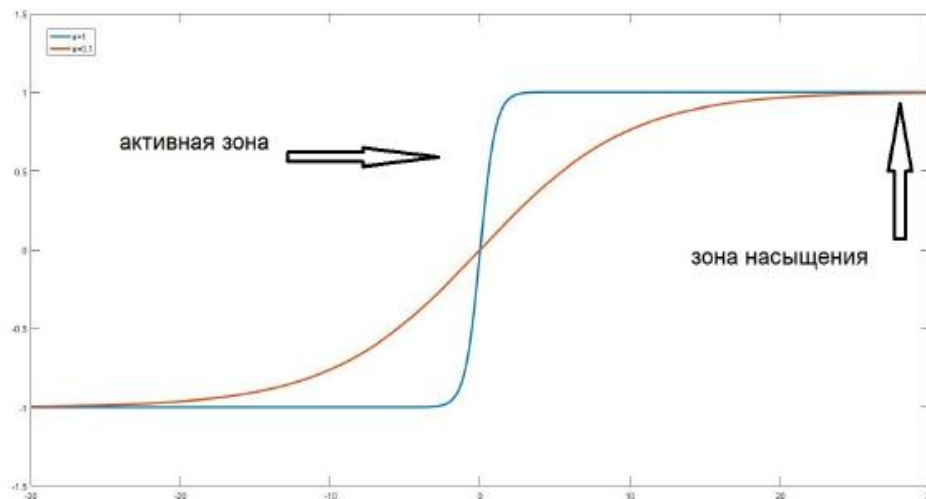
$$1 \cdot 0,79 + 1 \cdot 0,44 + 0 \cdot 0,43 = \text{sigma}(1,23) = 0,77$$

$$1 \cdot 0,85 + 1 \cdot 0,43 + 0 \cdot 0,29 = \text{sigma}(1,28) = 0,78$$

$$0,77 \cdot 0,5 + 0,78 \cdot 0,52 = \text{sigma}(0,79) = 0,69$$

ответ сети: 1 (идти)

Функция активации должна быть дифференцируема. Часто используется **сигмоида**:

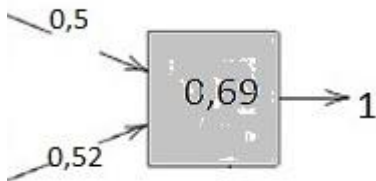


Сигмоида

$$S(z) = \frac{1}{1 + e^{-\alpha z}},$$

$$S'(z) = \alpha S(z) (1 - S(z));$$

Зная, что сеть нам сказала (идти), и то что мы хотим получить (не идти), можем приступить к этапу обучения сети. Подсчитаем ошибку и будем передавать ее справа на лево (обратное распространение ошибки) .



Откорректируем веса в выходном нейроне, который нам дал некорректный результат. Посчитаем ошибку – разницу между текущим результатом и ожидаемым:

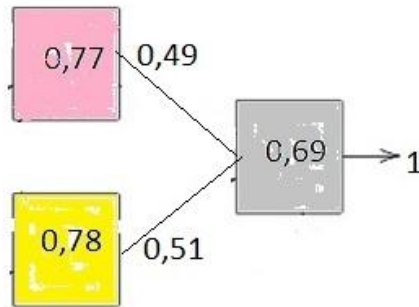
$error = 0,69 - 0 = 0,69$ (ошибка выходного слоя)

Считаем величину на которую нужно сдвинуть веса:

$$\Delta w = error * sigma'(x)$$

$$sigma'(x) = sigma(x)(1 - sigma(x)) = 0,69(1 - 0,69) = 0,21$$

$$\Delta w = 0,69 * 0,21 = 0,14$$



Введем параметр «скорость обучения сети» $\lambda = 0.1$

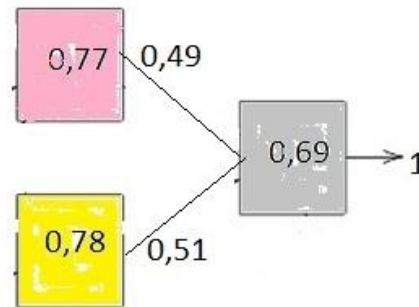
$$w_{1n} = w_1 - p_{21} \cdot \Delta \cdot \lambda = 0,5 - 0,77 * 0,14 * 0,1 = 0,49$$

$$w_{2n} = w_2 - p_{22} \cdot \Delta \cdot \lambda = 0,52 - 0,78 * 0,14 * 0,1 = 0,51$$

Дальше , считаем ошибку нейронов на скрытом слое. Эта ошибка неочевидна – характеризует внутреннее состояние сети. Т.е. ошибку отправляем на предыдущий уровень «назад»

$$error_1 = w_{1n} \cdot error = 0,49 * 0,14 = 0,07$$

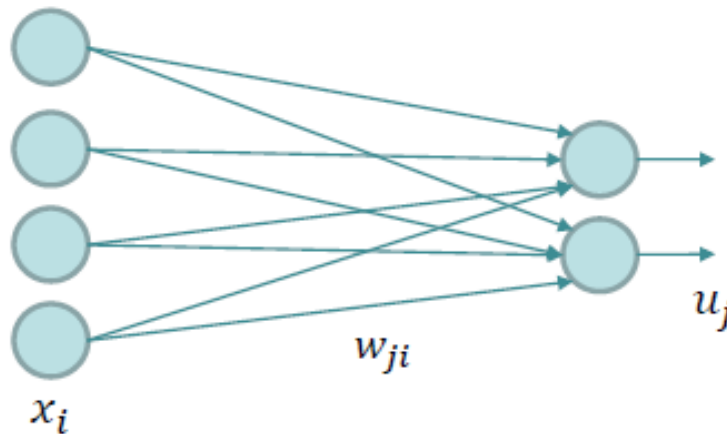
Далее , аналогично считаем ошибку для желтого нейрона и применяя выше перечисленные формулы корректируем веса на предыдущем слое.



А далее подбираем количество эпох η - это количество итераций обучения

Квадратичная функция ошибки для однослойной нейронной сети:

- Рассмотрим однослойную нейронную сеть, содержащую N входных и M выходных нейронов



- Обучающая выборка $\langle X, Y \rangle$:
 - L – число примеров в выборке,
 - X – множество входных сигналов (векторов размерности N),
 - Y – множество векторов фактических выходных сигналов (векторов размерности M)

Квадратичная функция ошибки для однослойной нейронной сети:

Квадратичная функции ошибки (используется для задачи восстановления регрессии):

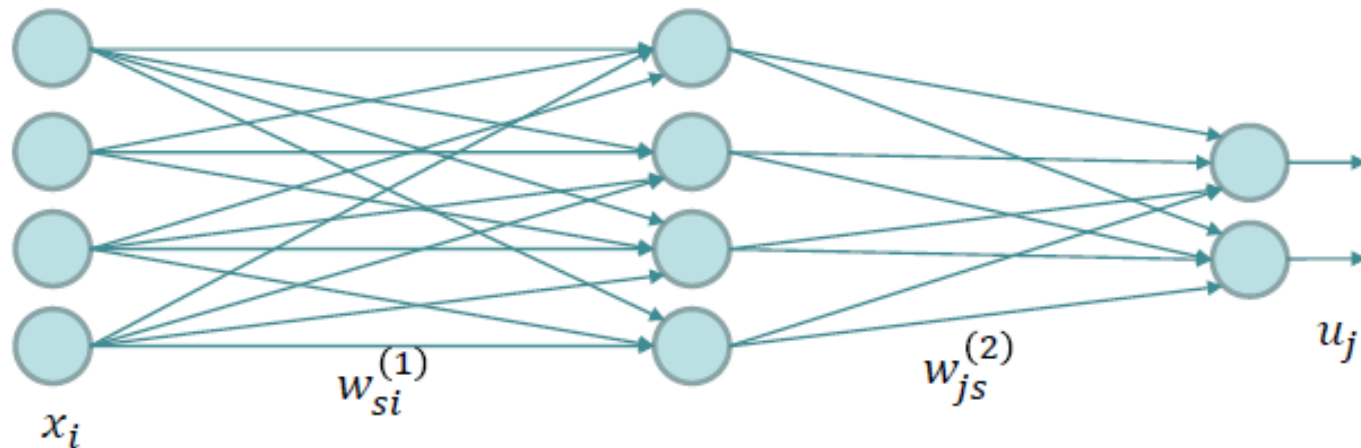
$$\begin{aligned} E &= \frac{1}{2} \sum_{k=1}^L \|y^k - u^k\|^2 = \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M (y_j^k - u_j^k)^2 \\ &= \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M \left(y_j^k - \varphi \left(\sum_{i=0}^N w_{ji} x_i^k \right) \right)^2, \end{aligned}$$

где $y^k = (y_j^k)_{j=\overline{1,M}} \in Y$ – множество обучающих примеров, а $u^k = (u_j^k)_{j=\overline{1,M}}$ – выход сети для входа $x^k = (x_i^k)_{i=\overline{1,N}} \in X$

Также вводится нормализация указанной метрики по числу примеров обучающей выборки L

Квадратичная функция ошибки для двухслойной нейронной сети:

Рассмотрим двухслойную нейронную сеть:



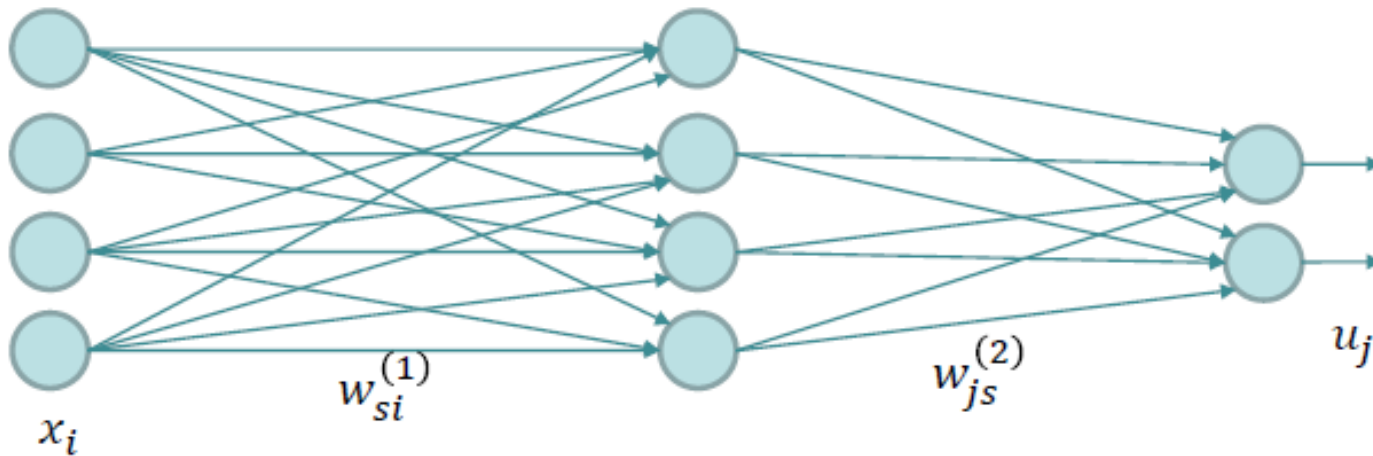
$w_{si}^{(1)}, w_{js}^{(2)}$ – веса синаптических связей

Выходной сигнал нейрона скрытого слоя описывается следующим образом:

$$v_s = \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right), s = \overline{0, K},$$
 где K – количество нейронов на скрытом слое

Квадратичная функция ошибки для двухслойной нейронной сети:

□ Рассмотрим двухслойную нейронную сеть:



□ Сигнал j -ого нейрона выходного слоя:

$$u_j = \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} v_s \right) = \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right), j = \overline{1, M}$$

Квадратичная функция ошибки для двухслойной нейронной сети:

- Квадратичная функция ошибки для набора тренировочных примеров имеет вид:

$$E(w) = \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M (y_j^k - u_j^k)^2$$
$$= \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M \left(y_j^k - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i^k \right) \right) \right)^2$$

Оптимизационная постановка задачи обучения с квадратичной функцией ошибки

- Общая математическая постановка задачи обучения сети с квадратичной функцией ошибки:

$$\min_w E(w) = \min_w \left\{ \frac{1}{L} \sum_{k=1}^L \left\{ \frac{1}{2} \sum_{j=1}^M (y_j^k - u_j^k)^2 \right\} \right\}$$

- Квадратичная ошибка отражает разницу выхода сети разметки
- Квадратичная (Евклидова) ошибка применяется при решении **задачи восстановления регрессии**

Оптимизационная постановка задачи обучения с кросс-энтропийной функцией ошибки

- Для **задачи классификации** (если входной объект относится только к одному классу!) в качестве функции ошибки выбирается **кросс-энтропия**. Постановка задачи обучения:

$$\min_w E(w) = \min_w \left\{ -\frac{1}{L} \sum_{k=1}^L \sum_{m=1}^M y_m^k \ln u_m^k \right\}$$

где $y_j^k = 1 \leftrightarrow x^k$ принадлежит классу j , иначе $y_j^k = 0$

- Является дифференцируемой аппроксимацией функции ошибки классификации «0-1»
- В качестве функции активации на последнем слое рекомендуется выбирать **функцию softmax**:

$$\varphi(u_j) = \frac{e^{u_j}}{\sum_{i=1}^M e^{u_i}}$$

Оптимизационная постановка задачи обучения с кросс-энтропийной функцией ошибки

- Для **задачи классификации** (если входной объект относится к нескольким классам!) в качестве функции ошибки выбирается **кросс-энтропия**. Постановка задачи обучения:

$$\min_w E(w) = \min_w \left\{ -\frac{1}{L} \sum_{k=1}^L \sum_{m=1}^M y_m^k \ln u_m^k + (1 - y_m^k) \ln(1 - u_m^k) \right\}$$

где $y_j^k = 1 \leftrightarrow x^k$ принадлежит классу j , иначе $y_j^k = 0$

Метод обратного распространения ошибки (Backpropagation)

- ❑ Метод обратного распространения ошибки определяет стратегию изменения параметров сети w в ходе обучения с использованием градиентных методов оптимизации
- ❑ Градиентные методы на каждом шаге уточняют значения параметров:

$$w(k + 1) = w(k) + \eta p(w)$$

- $\eta, 0 < \eta < 1$ – **скорость обучения** (learning rate) – «скорость» движения в направлении минимального значения функции,
- $p(w)$ – направление в многомерном пространстве параметров нейронной сети

- ❑ В классическом методе обратного распространения ошибки направление движения совпадает с направлением антиградиента $p(w) = -\nabla E(w)$



Метод обратного распространения ошибки (Backpropagation)

- ❑ **Инициализация синаптических весов сети**
(случайным образом из некоторого распределения)
- ❑ Повторение следующих шагов для каждого примера тренировочного набора данных
 - 1. Прямой проход:**
 1. Вычисление значений выходных сигналов нейронов всех слоев
 2. Вычисление значений производных функций активации на каждом слое сети
 - 2. Обратный проход:**
 1. Вычисление значения целевой функции и ее градиента
 2. Корректировка синаптических весов
- ❑ **Критерии остановки:** число итераций метода (количество проходов по всему множеству примеров), достигнутая ошибка

Алгоритм обратного распространения ошибки (Backpropagation)

Вход: $X^\ell = (x_i, y_i)_{i=1}^\ell$, $x_i \in \mathbb{R}^n$, $y_i \in \mathbb{R}^M$, параметры H, η, λ ;

Выход: веса w_{jh}, w_{hm} ;

1. инициализировать веса и текущую оценку $Q(w)$;

2. повторять

3. выбрать $(x_i, y_i)_{i=1}^\ell \in X^\ell$;

4. прямой ход:

$$u_i^h = \sigma_h \left(\sum_{j=0}^n w_{jh} x_i^j \right), \sigma'_h, h = 1, \dots, H;$$

$$a_i^m = \sigma_m \left(\sum_{h=0}^H w_{hm} u_i^h \right), \sigma'_m, \varepsilon_i^m = a_i^m - y_i, m = 1, \dots, M;$$

$$Q_i = \sum_{m=1}^M (\varepsilon_i^m)^2;$$

5. обратный ход:

$$\varepsilon_i^h = \sum_{m=1}^M \varepsilon_i^m \sigma'_m w_{hm}, \quad h = 1, \dots, H;$$

6. градиентный шаг:

$$w_{hm} = w_{hm} - \eta \varepsilon_i^m \sigma'_m u_i^h, \quad h = 0, \dots, H, \quad m = 1, \dots, M;$$

$$w_{jh} = w_{jh} - \eta \varepsilon_i^h \sigma'_h x_i^j, \quad j = 0, \dots, n, \quad h = 1, \dots, H;$$

7. оценка функционала:

$$Q = (1 - \lambda)Q + \lambda Q_i;$$

8. пока Q не стабилизируется;

Функции активации:

Должны быть дифференцируемы. Часто используются:

- Сигмоида

$$S(z) = \frac{1}{1 + e^{-\alpha z}},$$

$$S'(z) = \alpha S(z) (1 - S(z));$$

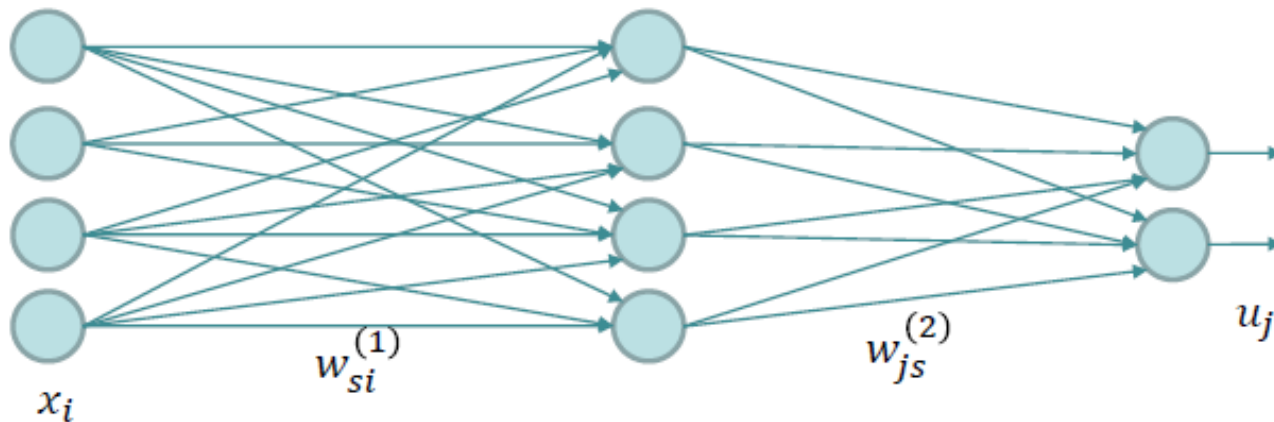
- Гиперболический тангенс

$$th(z) = \frac{sh(z)}{ch(z)} = \frac{e^{\alpha z} - e^{-\alpha z}}{e^{\alpha z} + e^{-\alpha z}} = \frac{1 - e^{-2\alpha z}}{1 + e^{-2\alpha z}} = \frac{2}{1 + e^{-2\alpha z}} - 1,$$

$$th'(z) = \alpha (1 - th^2(z)).$$

Метод обратного распространения ошибки на примере двухслойной сети

- Рассмотрим процедуру вычисления производных и значений функции ошибки на примере двухслойной нейронной сети

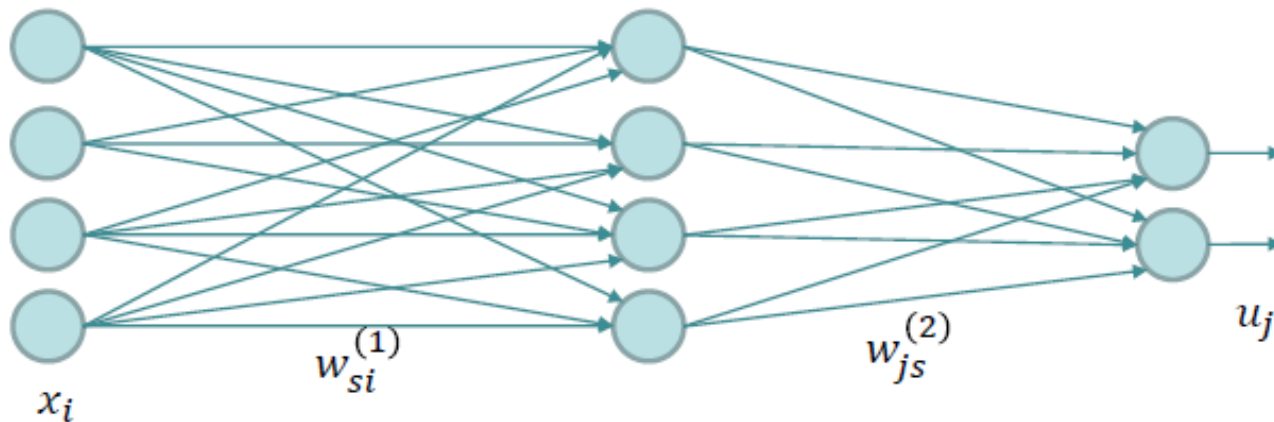


- Функция ошибки:

$$E(w) = \frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right) \right)^2$$

Метод обратного распространения ошибки на примере двухслойной сети

- Рассмотрим процедуру вычисления производных и значений функции ошибки на примере двухслойной нейронной сети



- Функция ошибки:

$$E(w) = \frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right) \right)^2$$

Метод обратного распространения ошибки на примере двухслойной сети

□ Производная целевой функции по параметрам последнего слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{js}^{(2)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \underbrace{\varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right)}_{u_j} \right)^2 \right)}{\partial w_{js}^{(2)}} \\ &= - \underbrace{(y_j - u_j)}_{\text{blue box}} \frac{d\varphi^{(2)}(g_j)}{dg_j} v_s = \delta_j^{(2)} v_s, \\ g_j &= \sum_{s=0}^K w_{js}^{(2)} v_s, \quad \delta_j^{(2)} = \frac{\partial E(w)}{\partial g_j} \end{aligned}$$

Производная целевой функции по параметрам последнего слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{js}^{(2)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \underbrace{\varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right)}_{v_s} \right)^2 \right)}{\partial w_{js}^{(2)}} \\ &= - \underbrace{(y_j - u_j)}_{\text{blue box}} \frac{d\varphi^{(2)}(g_j)}{dg_j} v_s = \delta_j^{(2)} v_s, \\ g_j &= \sum_{s=0}^K w_{js}^{(2)} v_s, \quad \delta_j^{(2)} = \frac{\partial E(w)}{\partial g_j} \end{aligned}$$



Производная целевой функции по параметрам последнего слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{js}^{(2)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\underbrace{\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right)}_{v_s} \right) \right)^2 \right)}{\partial w_{js}^{(2)}} \\ &= - (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} \underbrace{v_s}_{\text{blue box}} = \delta_j^{(2)} v_s, \\ g_j &= \sum_{s=0}^K w_{js}^{(2)} v_s, \quad \delta_j^{(2)} = \frac{\partial E(w)}{\partial g_j} \end{aligned}$$

Метод обратного распространения ошибки на примере двухслойной сети

Производная целевой функции по параметрам скрытого слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{si}^{(1)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right) \right)^2 \right)}{\partial w_{si}^{(1)}} \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} \frac{dg_j(v_s)}{dv_s} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} w_{js}^{(2)} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \delta_s^{(1)} x_i, \\ f_s &= \sum_{i=0}^N w_{si}^{(1)} x_i, \quad \delta_s^{(1)} = \frac{\partial E(w)}{\partial f_s} \end{aligned}$$



Производная целевой функции по параметрам скрытого слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{si}^{(1)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right) \right)^2 \right)}{\partial w_{si}^{(1)}} \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} \frac{dg_j(v_s)}{dv_s} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} w_{js}^{(2)} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \delta_s^{(1)} x_i, \\ f_s &= \sum_{i=0}^N w_{si}^{(1)} x_i, \quad \delta_s^{(1)} = \frac{\partial E(w)}{\partial f_s} \end{aligned}$$



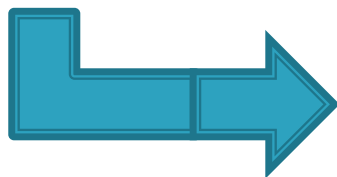
Производная целевой функции по параметрам скрытого слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{si}^{(1)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right) \right)^2 \right)}{\partial w_{si}^{(1)}} \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} \frac{dg_j(v_s)}{dv_s} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} w_{js}^{(2)} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \delta_s^{(1)} x_i, \\ f_s &= \sum_{i=0}^N w_{si}^{(1)} x_i, \quad \delta_s^{(1)} = \frac{\partial E(w)}{\partial f_s} \end{aligned}$$

Метод обратного распространения ошибки на примере двухслойной сети

Производная целевой функции по параметрам скрытого слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{si}^{(1)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right) \right)^2 \right)}{\partial w_{si}^{(1)}} \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} \frac{dg_j(v_s)}{dv_s} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} w_{js}^{(2)} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \delta_s^{(1)} x_i, \\ f_s &= \sum_{i=0}^N w_{si}^{(1)} x_i, \quad \delta_s^{(1)} = \frac{\partial E(w)}{\partial f_s} \end{aligned}$$

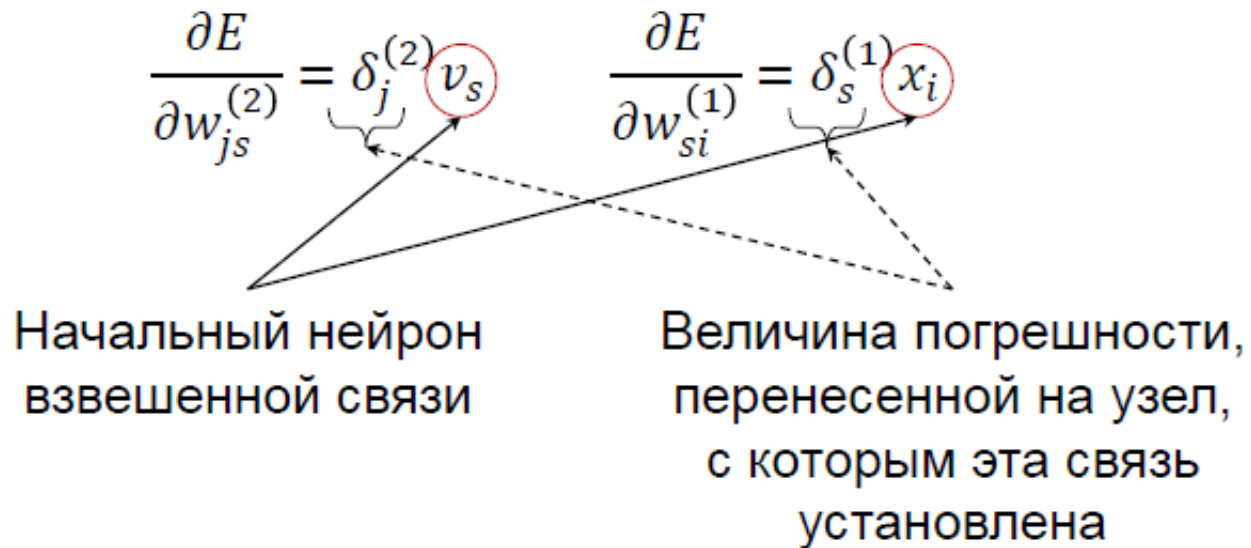


Производная целевой функции по параметрам скрытого слоя:

$$\begin{aligned} \frac{\partial E}{\partial w_{si}^{(1)}} &= \frac{\partial \left(\frac{1}{2} \sum_{j=1}^M \left(y_j - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right) \right)^2 \right)}{\partial w_{si}^{(1)}} \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} \frac{dg_j(v_s)}{dv_s} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \\ &= - \sum_{j=1}^M (y_j - u_j) \frac{d\varphi^{(2)}(g_j)}{dg_j} w_{js}^{(2)} \frac{d\varphi^{(1)}(f_s)}{df_s} x_i = \delta_s^{(1)} x_i, \\ f_s &= \sum_{i=0}^N w_{si}^{(1)} x_i, \quad \delta_s^{(1)} = \frac{\partial E(w)}{\partial f_s} \end{aligned}$$

Метод обратного распространения ошибки на примере двухслойной сети

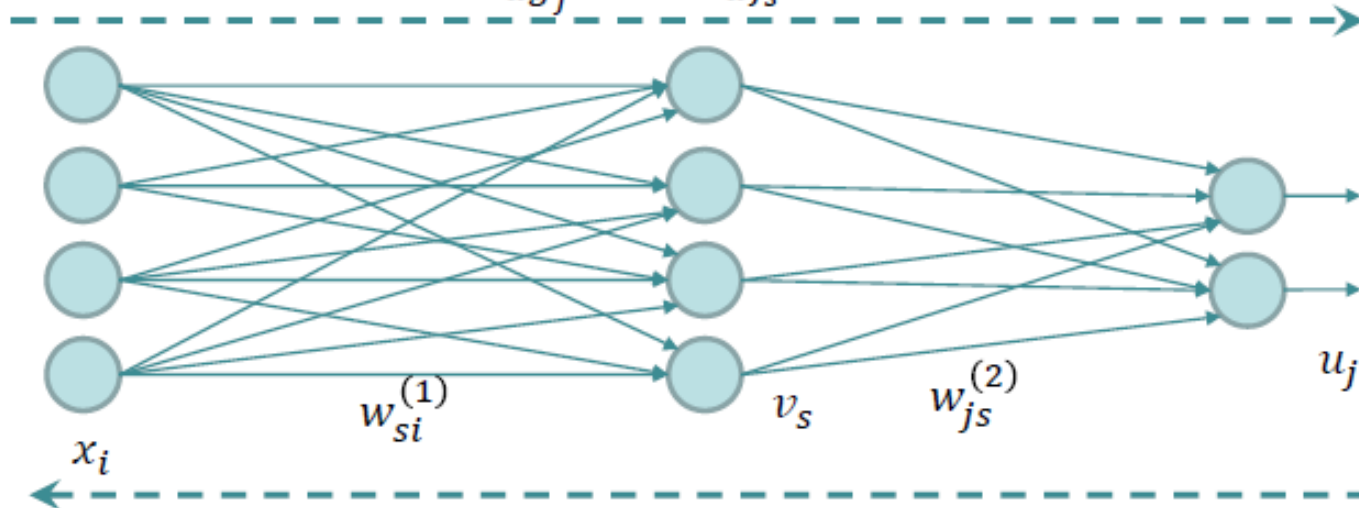
- Описание градиента по параметрам выходного и скрытого слоев имеют идентичную структуру:



Метод обратного распространения ошибки на примере двухслойной сети

1. Прямой проход:

- Вычисление v_s и u_j
- Вычисление $\frac{d\varphi^{(2)}(g_j)}{dg_j}$ и $\frac{d\varphi^{(1)}(f_s)}{df_s}$



2. Обратный проход:

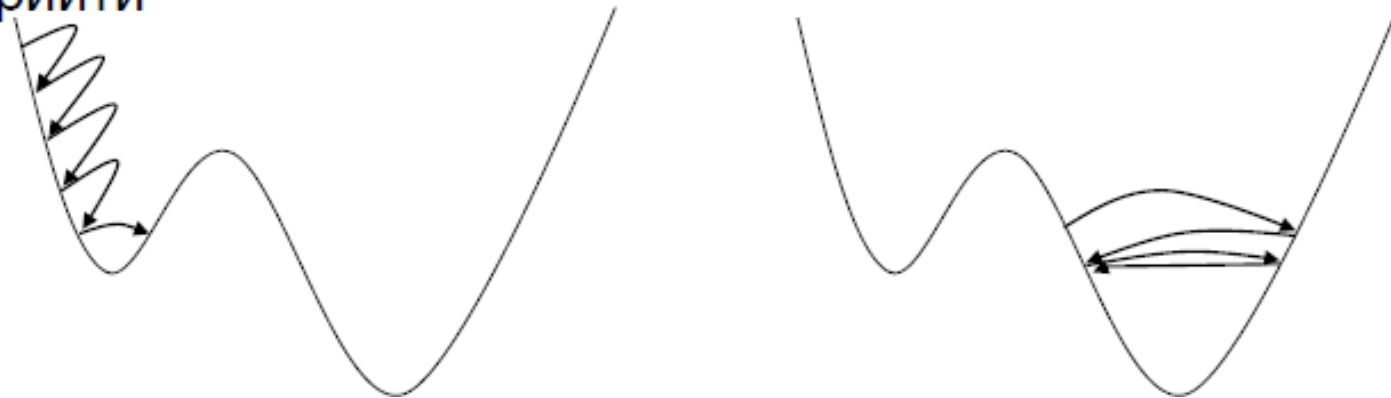
- Вычисление целевой функции E и ее градиента $\frac{\partial E}{\partial w_{js}^{(2)}}$ и $\frac{\partial E}{\partial w_{si}^{(1)}}$
- Коррекция весов $w(k+1) = w(k) - \eta \nabla E(w)$

Выбор скорости обучения

Скорость обучения определяет размер шага, с которым будет происходить движение в сторону минимума

Проблемы выбора скорости обучения:

- Если шаг маленький, то обучение будет происходить медленно и есть возможность «застрять» в локальном минимуме
- Если шаг слишком большой, то можно бесконечно «прыгать» через точку минимума, но никогда в нее не прийти



Проблемы алгоритма обратного распространения ошибки

- ▶ **Паралич сети.** В процессе обучения сети значения весов связей могут, в результате коррекции, стать очень большими величинами. Это может привести к тому, что все или большинство нейронов будут функционировать при больших значениях аргумента активационной функции, где производная активационной функции очень мала. Поскольку посылаемая обратно в процессе обучения ошибка пропорциональна этой производной, процесс обучения может практически остановиться. Обычно этого избегают уменьшением коэффициента скорости обучения η , но в таком случае увеличивается время обучения.

Чтобы ограничить рост весов, добавим к функционалу $Q(w)$ штрафное слагаемое:

$$Q_{\tau}(w) = Q(w) + \frac{\tau}{2} \|w\|^2.$$

Тогда у градиента появится добавка:

$$\frac{\partial Q_{\tau}(w)}{\partial w} = \frac{\partial Q(w)}{\partial w} + \tau w.$$

Правило обновления весов теперь примет вид:

$$w = w - \eta \frac{\partial Q_{\tau}(w)}{\partial w} = w(1 - \eta\tau) - \eta \frac{\partial Q(w)}{\partial w}.$$

Множитель $1 - \eta\tau$ приводит к постоянному уменьшению весов.

Нормализация данных

Диапазоны различных признаков могут сильно отличаться, это затрудняет процесс обучения.

Два способа нормализации признаков:

$$x^j = \frac{x^j - x_{min}^j}{x_{max}^j - x_{min}^j},$$

$$x^j = \frac{x^j - x_{cp}^j}{x_{cko}^j},$$

где x_{min}^j , x_{max}^j , x_{cp}^j , x_{cko}^j – соответственно минимальное, максимальное, среднее значения и среднеквадратичное отклонение j -ого признака.

Инициализация весов

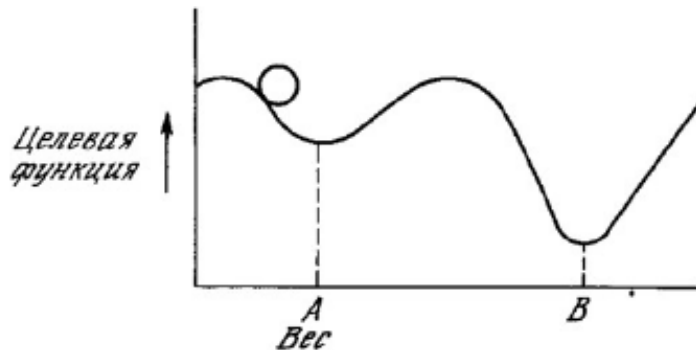
- нулевые значения
- случайные небольшие значения
- значения $w_j = \frac{\langle y, f_j \rangle}{\langle f_j, f_j \rangle}$, где $f_j = (f_j(x_i))_{i=1}^{\ell}$
 $\langle y, f_j \rangle$ - корреляция между выходом и j -ым признаком -
чем они теснее связаны, тем больше должен быть вектор
весов при j -ом признаке,
формулы точны в случае, когда функция потерь
квадратична и признаки статистически независимы

Порядок предъявления объектов

- случайный порядок
- перетасовка объектов - попеременно предъявлять сети объекты из разных классов (shuffling)
- чаще предъявлять те объекты, на которых была допущена большая ошибка (если исходные данные не содержат выбросов, иначе процесс обучения сосредоточится на шумовых объектах, которые вообще следовало бы исключить из обучающей выборки)

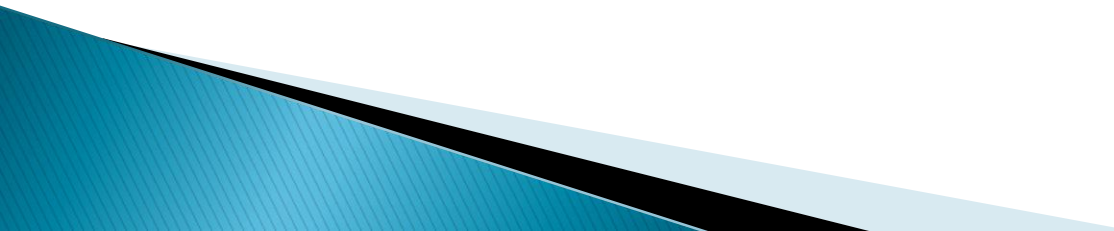
- **Попадание в локальные минимумы.** Функция ошибки сложной сети сильно изрезана и состоит из холмов, долин, складок и оврагов в пространстве высокой размерности. Сеть может попасть в локальный минимум (неглубокую долину), когда рядом имеется гораздо более глубокий минимум.

Функционал ошибки $Q(w)$ является многоэкстремальным и часто в процессе обучения попадает в локальный минимум, где и стабилизируется до некоторого высокого для нас значения.



Log of weights (встряска, подталкивание) – метод, заключающийся в случайной модификации вектора весов после стабилизации функционала и запуске градиентного процесса из новой точки.

Критерий остановки

- Функционал качества на контрольной выборке начинает расти, а на обучении – падать.
Сеть настраивается на объекты обучающей выборки, а ее обобщающая способность снижается (переобучение сети).
 - Функционал ошибки достиг установленного минимального значения.
 - Достигнуто определенное количество итераций.
 - Функционал ошибки стабилизировался.
- 

Диагональный метод Левенберга-Марквардта

Метод Ньютона-Рафсона (второго порядка):

$$w = w - \eta (\mathcal{L}_i''(w))^{-1} \mathcal{L}_i'(w),$$

где $\mathcal{L}_i''(w) = \left(\frac{\partial^2 \mathcal{L}_i(w)}{\partial w_{jh} \partial w_{j'h'}} \right)$ – гессиан порядка $(n+1)H + (H+1)M$.

Эвристика. Считаем, что гессиан диагонален:

$$w_{jh} = w_{jh} - \eta \left(\frac{\partial^2 \mathcal{L}_i(w)}{\partial w_{jh}^2} + \mu \right)^{-1} \frac{\partial \mathcal{L}_i(w)}{\partial w_{jh}},$$

η – темп обучения, μ – параметр, предотвращающий обнуление знаменателя.

Bagging (bootstrap aggregation) Bremen 1996

– более тонкий способ формирования начального приближения.

Идея – сначала обучить нейроны первого скрытого слоя на различных случайных bootstrap-выборках, как H однослойных персептронов простейшими градиентными методами. Затем по отдельности настроить нейроны второго слоя, которым на вход подается вектор выходных значений первого слоя и т.д.

Процедура формирования bootstrap-выборок из объектов обучающей выборки называется bootstrapping, объединение отдельно настроенных нейронов в сеть - aggregation.

Последовательное наращивание

- сеть обучается при заведомо недостаточном количестве нейронов, пока ошибка не перестанет убывать;
- добавляется новый нейрон со случайными весами или уже обученный отдельно (по случайной подвыборке либо по объектам с наибольшим значением потерь);
- продолжается обучение сети.

Время обучения обычно в 1,5-2 раза больше, чем если бы в сети сразу было нужное количество нейронов.

Добиться такого же малого значения ошибки как при динамическом наращивании сети, взяв сразу сеть нужной структуры, не удастся.

Прореживание сети OBD (Optimal brain damage)

Аппроксимируем $Q(w)$ в локальном минимуме w квадратичной формой:

$$Q(w + \delta) = Q(w) + \frac{1}{2} \delta^T Q''(w) \delta + o(\|\delta\|^2),$$

где $Q''(w) = \left(\frac{\partial^2 Q(w)}{\partial w_{jh} \partial w_{j'h'}} \right)$ – гессиан порядка $(n+1)H + (H+1)M$.

Эвристика. Пусть гессиан $Q''(w)$ диагонален, тогда

$$\delta^T Q''(w) \delta = \sum_{j=0}^n \sum_{h=1}^H \delta_{jh}^2 \frac{\partial^2 Q(w)}{\partial w_{jh}^2} + \sum_{h=0}^H \sum_{m=1}^M \delta_{hm}^2 \frac{\partial^2 Q(w)}{\partial w_{hm}^2}$$

Обнуляем вес: $w_{jh} + \delta_{jh} = 0$.

Значимостью (salience) веса w_{jh} называется изменение

функционала $Q(w)$ при обнулении веса: $S_{jh} = w_{jh}^2 \frac{\partial^2 Q(w)}{\partial w_{jh}^2}$.

- в BackProp вычислять вторые производные $\frac{\partial^2 Q(w)}{\partial w_{jh}^2}$, $\frac{\partial^2 Q(w)}{\partial w_{hm}^2}$;
- если процесс минимизации $Q(w)$ пришел в минимум, то
 - упорядочить все веса по убыванию S_{jh} ;
 - удалить N связей с наименьшей значимостью;
 - снова запустить BackProp;
- если $Q(w, X^\ell)$ или $Q(w, X^k)$ существенно ухудшился, то вернуть последние удаленные связи и выйти

Отбор признаков при помощи OBD – аналогично. Суммарная значимость признака: $S_j = \sum_{h=1}^H S_{jh}$.

OBS (Optimal Brain Surgery) – учитывает все компоненты гессиана, вычислительно сложнее. OBD – частный случай OBS.

Задачи классификации

В задаче классификации имеется множество объектов, разделённых некоторым образом на классы.

Требуется классифицировать произвольный объект, т.е. указать номер или наименование класса, к которому он относится.

Сами классы известны, обучение проходит с учителем.

- Задачи медицинской и технической диагностики
- Задача распознавания рукописных индексов
- Задача оценивания заемщиков банком
- ...

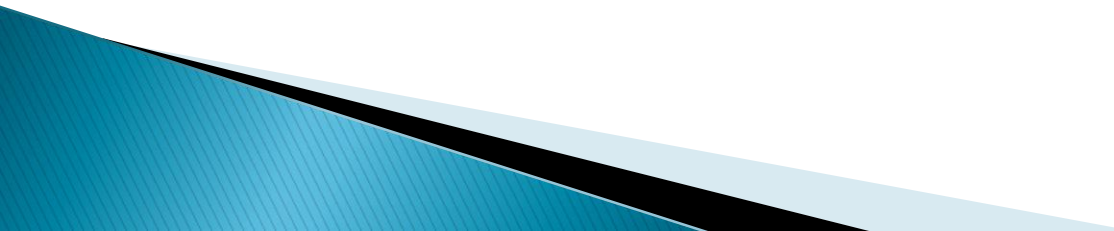
Ирисы Фишера

– набор данных о 150 экземплярах ириса, по 50 экземпляров каждого из трёх видов. Для каждого экземпляра измерялись:

- Длина чашелистика (sepal length);
- Ширина чашелистика (sepal width);
- Длина лепестка (petal length);
- Ширина лепестка (petal width).

Требуется построить правило классификации, определяющее вид растения по данным измерений.

Класс *Iris setosa* линейно-разделим от двух остальных.



Ирис щетинистый (*Iris setosa*)



Ирис Виргинский (*Iris virginica*)



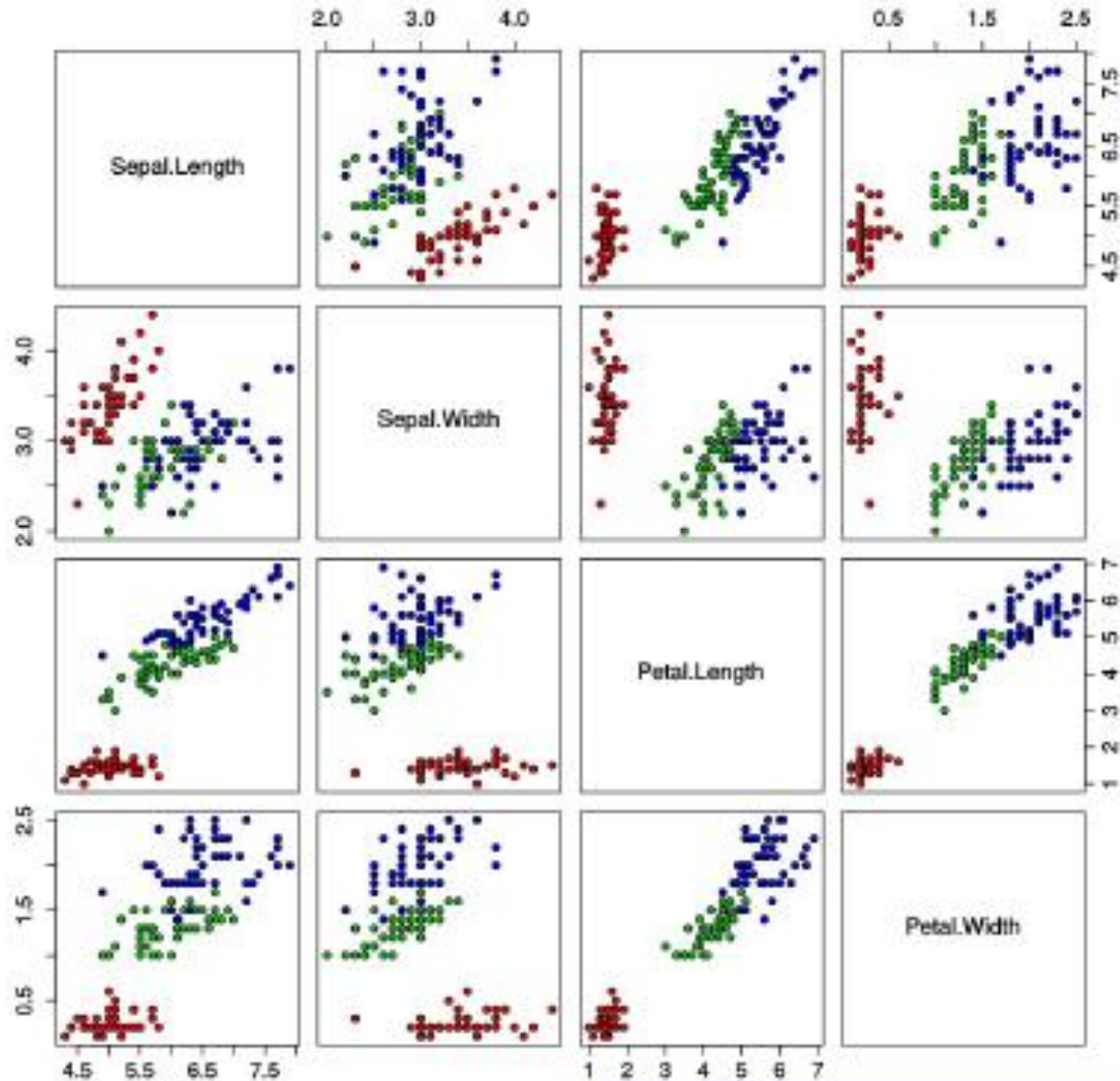
Ирис разноцветный (*Iris versicolor*)



Ирисы Фишера

Длина чашелистика ⇄	Ширина чашелистика ▲	Длина лепестка ⇄	Ширина лепестка ⇄	Вид ириса ⇄
5.0	2.0	3.5	1.0	<i>versicolor</i>
6.0	2.2	4.0	1.0	<i>versicolor</i>
6.2	2.2	4.5	1.5	<i>versicolor</i>
6.0	2.2	5.0	1.5	<i>virginica</i>
4.5	2.3	1.3	0.3	<i>setosa</i>
5.5	2.3	4.0	1.3	<i>versicolor</i>
6.3	2.3	4.4	1.3	<i>versicolor</i>
5.0	2.3	3.3	1.0	<i>versicolor</i>
4.9	2.4	3.3	1.0	<i>versicolor</i>
5.5	2.4	3.8	1.1	<i>versicolor</i>
5.5	2.4	3.7	1.0	<i>versicolor</i>
5.6	2.5	3.9	1.1	<i>versicolor</i>
6.3	2.5	4.9	1.5	<i>versicolor</i>
5.5	2.5	4.0	1.3	<i>versicolor</i>

Iris Data (red=setosa,green=versicolor,blue=virginica)



Задача кластеризации

В задаче кластеризации имеется множество объектов, их нужно разделить на классы (обособленные «сгустки» объектов) и научиться относить новые объекты к этим кластерам.

